

Process Synchronization

RECAP:

Memory mapping means associating a file or device with a portion of a process's virtual memory space. It allows your program to access file data as if it were in memory (RAM).

mmap () takes file and map it directly into process's memory space.

- Reserve a block of your virtual memory (length bytes).
- Connect (map) that memory block to the contents of a file (fd) starting at offset.
- Set access permissions (prot), like read/write/execute.
- Return a pointer to that memory area, you can now directly read/write bytes.

munmap() Unmap a mapped region

mremap() Resize mapped memory

memcpy() modifies mapped memory

- [memory mapping](#)
- [mmap manual](#)
- [Extend or shrink existing memory \(mremap\)](#)
- [Memory mapping, translate logical address to physical address](#)

1. What Are Named Semaphores?

A **named semaphore** is a semaphore that has a name just like a file has a name in the filesystem. This name allows the semaphore to be accessed by any process on the system, not just processes that are related to each other (parent-child). In simpler terms: a named semaphore is a system-wide lock that any process can find by its name, open, and use for synchronization even if the processes were started completely independently.

Core Concept

Named semaphores live in the kernel and are identified by a string name (e.g. /sem_ping). Any process that knows the name can open and use the semaphore, no shared ancestry required.

1.1 How are they created?

Named semaphores are created and opened using `sem_open()`:

```
#include <semaphore.h>

#include <fcntl.h>

sem_t *sem = sem_open("/my_semaphore", // Name (must start with /)
                    O_CREAT | O_EXCL, // Create; fail if already exists
                    0666,             // Permissions (like a file)
                    1);                // Initial value
```

1.2 The lifecycle of a named semaphore.

Step	Function	What It Does
Create	<code>sem_open(name, O_CREAT, ...)</code>	Creates a new named semaphore in the kernel

Step	Function	What It Does
Open	<code>sem_open(name, 0)</code>	Opens an existing named semaphore by name
Wait	<code>sem_wait(sem)</code>	Decrements (locks); blocks if value is 0
Signal	<code>sem_post(sem)</code>	Increments (unlocks); wakes a waiting process
Close	<code>sem_close(sem)</code>	Closes the handle in this process (still exists)
Delete	<code>sem_unlink(name)</code>	Removes the semaphore from the kernel permanently

2. Why Use Named Semaphores?

Named semaphores solve a specific problem: **synchronization between completely unrelated processes**. They are the right tool when the processes you want to synchronize were not created from the same parent process using `fork()`.

2.1 Typical Use Cases:

- **Server–Client Programs:** A web server and a logging daemon started independently need to coordinate file writes.
- **Independent Executables:** Two separate programs (ping and pong) compiled and run as separate binaries.
- **System Services:** A system daemon and a user application sharing access to a hardware resource.
- **Named Pipe Alternatives:** When you need event signaling rather than data transfer between unrelated processes.

2.2 What problems do they solve?

The Core Problem

If Process A and Process B are started from separate terminals or scripts, they share no memory. They cannot use `sem_init()` with a shared memory pointer because they don't know each other's addresses. Named semaphores solve this — the kernel holds the semaphore, and the name is the shared reference.

3. Named vs Unnamed Semaphores

This is one of the most important distinctions to understand. The two kinds of semaphores serve different purposes and are used in different situations.

Property	Named (<code>sem_open</code>)	Unnamed (<code>sem_init</code> + <code>mmap</code>)
Creation	<code>sem_open("/name", O_CREAT, ...)</code>	<code>sem_init(sem, 1, value)</code>
Identification	String name in kernel namespace	Memory address (pointer)
Scope	Any process; system-wide	Only <code>fork()</code> : related processes
Persistence	Survives process exit (until unlink)	Disappears when process exits
Cleanup	<code>sem_close()</code> + <code>sem_unlink()</code>	<code>sem_destroy()</code> + <code>munmap()</code>
Storage	Kernel (not in process memory)	Shared memory region (<code>mmap</code>)

Property	Named (sem_open)	Unnamed (sem_init + mmap)
Use Case	Separate/unrelated processes	Parent + children from fork()
Shared Memory?	NOT required	REQUIRED (mmap or similar)

3.1 Named Semaphores:

The operating system stores the semaphore in its own internal space (the kernel). It assigns a name to it. Any process from any terminal, any user (with permission) can call `sem_open("/name", 0)` to get a handle to that same semaphore.

```
// Process A (e.g. ping.c)

sem_t *sem_ping = sem_open("/sem_ping", O_CREAT, 0666, 1);
sem_t *sem_pong = sem_open("/sem_pong", O_CREAT, 0666, 0);

// Process B (e.g. pong.c) — completely separate binary

sem_t *sem_ping = sem_open("/sem_ping", 0); // Open existing
sem_t *sem_pong = sem_open("/sem_pong", 0); // Open existing
```

Why Named Semaphores Don't Need Shared Memory

```
// ping.c — Process A (separate executable, no shared memory setup)

sem_t *sem_ping = sem_open("/sem_ping", O_CREAT | O_EXCL, 0666, 1);
sem_t *sem_pong = sem_open("/sem_pong", O_CREAT | O_EXCL, 0666, 0);

sem_wait(sem_ping);    // Blocks until kernel says value > 0
printf("ping\n");

sem_post(sem_pong);    // Kernel increments sem_pong — wakes pong.c

// pong.c — Process B (another separate executable)

sem_t *sem_ping = sem_open("/sem_ping", 0); // Find by name
sem_t *sem_pong = sem_open("/sem_pong", 0); // Find by name

sem_wait(sem_pong);    // Blocks until ping.c posts
printf("pong\n");

sem_post(sem_ping);    // Wakes ping.c
```

Notice there is no `mmap()`, no `shm_open()`, no shared buffer of any kind. The two programs synchronize **purely through the kernel**.

When You Also Need Shared Memory?

Shared memory is needed when processes need to **share data** not just signals. In ping.c / pong.c example with shared_data_t, the counter variable lives in shared memory so both processes can read and write it:

```
// Both processes do this to access the shared counter:

int shm_fd = shm_open("/shm_pingpong", O_RDWR, 0666);

shared_data_t *data = mmap(NULL, sizeof(shared_data_t),
                           PROT_READ|PROT_WRITE, MAP_SHARED, shm_fd, 0);

data->counter++; // Both processes can read/write this counter

// The SEMAPHORES still don't need this memory.

// Shared memory is ONLY for the counter data.
```

Scenario	Shared Memory Needed?	Semaphore Type
Synchronize unrelated processes (signals only)	No	Named (sem_open)
Synchronize + share data between unrelated processes	Yes (for data)	Named (sem_open)
Synchronize parent + fork()-children (signals only)	Yes (for sem_t)	Unnamed (sem_init + mmap)
Synchronize parent + fork()-children + share data	Yes (for both)	Unnamed (sem_init + mmap)

Examples:

Named Semaphores without shared memory:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <semaphore.h>
#include <sys/wait.h>

#define SEM_PING "/sem_ping"
#define SEM_PONG "/sem_pong"
#define ITERATIONS 20

int main() {
    // Create two named semaphores
    sem_t *sem_ping = sem_open(SEM_PING, O_CREAT | O_EXCL, 0666, 1); // ping starts first
    sem_t *sem_pong = sem_open(SEM_PONG, O_CREAT | O_EXCL, 0666, 0); // pong waits

    // Clean up if semaphores already existed
```

```

if (sem_ping == SEM_FAILED || sem_pong == SEM_FAILED) {
    sem_unlink(SEM_PING);
    sem_unlink(SEM_PONG);
    sem_ping = sem_open(SEM_PING, O_CREAT | O_EXCL, 0666, 1);
    sem_pong = sem_open(SEM_PONG, O_CREAT | O_EXCL, 0666, 0);
}

pid_t ping_pid = fork();
if (ping_pid == 0) {
    for (int i = 0; i < ITERATIONS; i++) {
        sem_wait(sem_ping);
        printf("ping\n");
        fflush(stdout);
        sem_post(sem_pong);
    }
    exit(0);
}

pid_t pong_pid = fork();
if (pong_pid == 0) {
    for (int i = 0; i < ITERATIONS; i++) {
        sem_wait(sem_pong);
        printf("pong\n");
        fflush(stdout);
        sem_post(sem_ping);
    }
    exit(0);
}

waitpid(ping_pid, NULL, 0);
waitpid(pong_pid, NULL, 0);

sem_close(sem_ping);
sem_close(sem_pong);
sem_unlink(SEM_PING);
sem_unlink(SEM_PONG);

return 0;
}

```

Unnamed Semaphores with mmap

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>

#define ITERATIONS 20

int main() {
    // Create unnamed semaphore in shared memory
    sem_t *sem_ping = mmap(NULL, sizeof(sem_t), PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    sem_t *sem_pong = mmap(NULL, sizeof(sem_t), PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS, -1, 0);
}

```

```

if (sem_ping == MAP_FAILED || sem_pong == MAP_FAILED) {
    perror("mmap");
    exit(1);
}

// Initialize semaphores: ping starts with 1, pong waits with 0
sem_init(&sem_ping, 1, 1); // 1 = shared between processes
sem_init(&sem_pong, 1, 0);

pid_t ping_pid = fork();
if (ping_pid == 0) {
    for (int i = 0; i < ITERATIONS; i++) {
        sem_wait(&sem_ping);
        printf("ping\n");
        fflush(stdout);
        sem_post(&sem_pong);
    }
    exit(0);
}

pid_t pong_pid = fork();
if (pong_pid == 0) {
    for (int i = 0; i < ITERATIONS; i++) {
        sem_wait(&sem_pong);
        printf("pong\n");
        fflush(stdout);
        sem_post(&sem_ping);
    }
    exit(0);
}

waitpid(ping_pid, NULL, 0);
waitpid(pong_pid, NULL, 0);

sem_destroy(&sem_ping);
sem_destroy(&sem_pong);
munmap(sem_ping, sizeof(sem_t));
munmap(sem_pong, sizeof(sem_t));

return 0;
}

```

Here we create an anonymous shared memory region. That memory:

- Is only shared between parent and children created using fork().
- Cannot be accessed by totally separate processes (like separate binaries/files run independently).
- There's no persistent name or file backing it — once the process ends, the memory vanishes.

So this only works because all processes are created from one executable using fork().

Shared Memory & Semaphores

In this example shared data is not actually used, we can print ping, pong alternatively without it. I just added this to show how we can work with shared memory and semaphores.

Shared data: shared_def.h

```

#ifndef SHARED_DEFS_H
#define SHARED_DEFS_H

```

```
#define SHM_NAME "/shm_pingpong"
#define SEM_PING "/sem_ping"
#define SEM_PONG "/sem_pong"
#define ITERATIONS 20

typedef struct {
    int counter;
} shared_data_t;

#endif
```

Ping.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <sys/stat.h>
#include "shared_defs.h"

int main() {
    // Create or open shared memory
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(shared_data_t));
    shared_data_t *shared_data = mmap(NULL, sizeof(shared_data_t), PROT_READ | PROT_WRITE,
    MAP_SHARED, shm_fd, 0);

    // Initialize counter only if it's the first run
    shared_data->counter = 0;

    sem_t *sem_ping = sem_open(SEM_PING, O_CREAT, 0666, 1);
    sem_t *sem_pong = sem_open(SEM_PONG, O_CREAT, 0666, 0);

    for (int i = 0; i < ITERATIONS; i++) {
        sem_wait(sem_ping);
        printf("ping\n");
        fflush(stdout);
        shared_data->counter++;
        sem_post(sem_pong);
    }

    munmap(shared_data, sizeof(shared_data_t));
    close(shm_fd);
    sem_close(sem_ping);
    sem_close(sem_pong);

    return 0;
}
```

Pong.c

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <sys/stat.h>
#include "shared_defs.h"

int main() {
    // Open existing shared memory
    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    shared_data_t *shared_data = mmap(NULL, sizeof(shared_data_t), PROT_READ | PROT_WRITE,
MAP_SHARED, shm_fd, 0);

    sem_t *sem_ping = sem_open(SEM_PING, 0);
    sem_t *sem_pong = sem_open(SEM_PONG, 0);

    for (int i = 0; i < ITERATIONS; i++) {
        sem_wait(sem_pong);
        printf("pong\n");
        fflush(stdout);
        shared_data->counter++;
        sem_post(sem_ping);
    }

    munmap(shared_data, sizeof(shared_data_t));
    close(shm_fd);
    sem_close(sem_ping);
    sem_close(sem_pong);

    if (shared_data->counter >= ITERATIONS * 2) {
        shm_unlink(SHM_NAME);
        sem_unlink(SEM_PING);
        sem_unlink(SEM_PONG);
    }

    return 0;
}

```